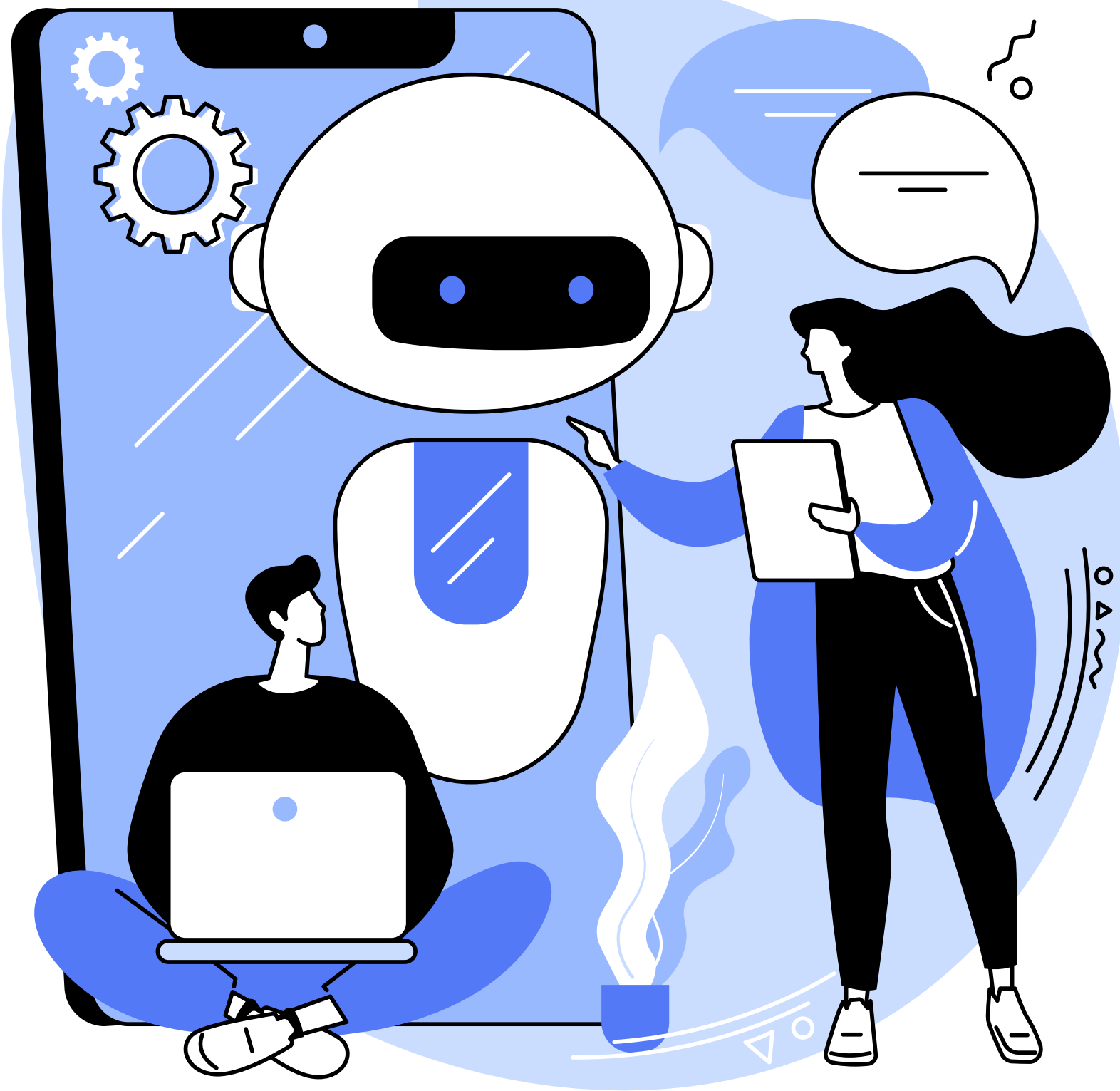




MobileNet Google Colab

☰

🔍

⏪

🔑

📁

📊

Files

📁 ..

📁 sample_data

📁

✕

⬆️

🔄

📁

🔍

[1]

✓ Os

▶

1 !nvidia-smi

2

⌵

⋮

Mon Dec 8 15:28:50 2025

+-----+-----+-----+				NVIDIA-SMI 550.54.15				Driver Version: 550.54.15				CUDA Version: 12.4				+-----+-----+-----+			
GPU		Name		Persistence-M		Bus-Id		Disp.A		Volatile		Uncorr. ECC		GPU-Util		Compute M.		ECC	
Fan		Temp		Perf		Pwr:Usage/Cap		Memory-Usage		GPU-Util		Compute M.		MIG M.					
=====																			
0		NVIDIA A100-SXM4-40GB		Off		00000000:00:04.0		Off								0			
N/A		32C		P0		47W / 400W		0MiB / 40960MiB						0%		Default		Disabled	
+-----+-----+-----+																			
Processes:																			
GPU		GI		CI		PID		Type		Process name				GPU Memory					
		ID		ID										Usage					
=====																			
No running processes found																			
+-----+-----+-----+																			

GPU modeli: NVIDIA A100-SXM4-40GB

CUDA driver: 550.54.15

A100 (Google Colab) vs V100 (Akya CUDA)

For training convnets with PyTorch, the Tesla A100 is...

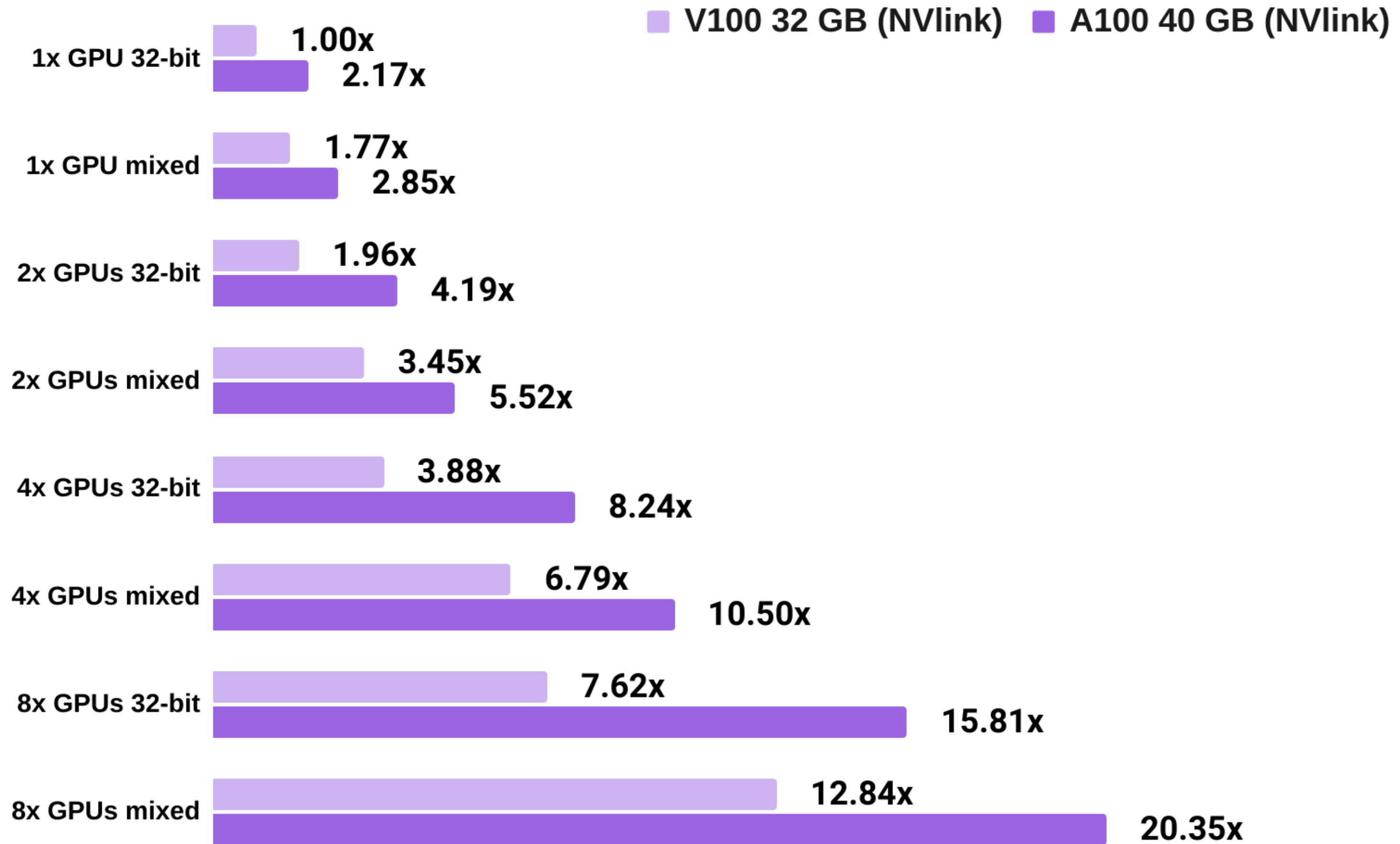
- + **2.2x** faster than the V100 using 32-bit precision.*
- + **1.6x** faster than the V100 using mixed precision.

For training language models with PyTorch, the Tesla A100 is...

- + **3.4x** faster than the V100 using 32-bit precision.
- + **2.6x** faster than the V100 using mixed precision.

Source: <https://lambda.ai/blog/nvidia-a100-vs-v100-benchmarks>

A100 vs V100 convnet training speed, PyTorch



[]

```
1 !pip install nvidia-pyindex
2 !pip install tensorrt==10.0.1
3 !pip install onnx onnxruntime
4
```

- **TensorRT Python API aktif hale gelir**
- **ONNX modeli Colab ortamında okunabilir**
- **TRT engine oluşturulabilir**
- **Nsight profiling sırasında kaydedilecek GPU kernel operatörleri artık doğru şekilde çalışabilir**

ONNX File:

<https://huggingface.co/qualcomm/MobileNet-v2/blob/2845d744eed4da1f7ae54958372ccba079482b0d/MobileNet-v2.onnx>



```
1 import tensorrt as trt
2
3 onnx_file = "MobileNet-v2.onnx"
4 engine_file = "MobileNet-v2.engine"
5
6 logger = trt.Logger(trt.Logger.INFO)
7 builder = trt.Builder(logger)
8
9 # TRT10: EXPLICIT_BATCH flag
10 explicit_batch_flag = 1 << int(trt.NetworkDefinitionCreationFlag.EXPLICIT_BATCH)
11 network = builder.create_network(explicit_batch_flag)
12
13 parser = trt.OnnxParser(network, logger)
14
15 print("Parsing ONNX...")
16 with open(onnx_file, "rb") as f:
17     if not parser.parse(f.read()):
18         print("ONNX PARSE FAILED")
19         for i in range(parser.num_errors):
20             print(parser.get_error(i))
21         raise SystemExit
22
23 # TRT 10.14 config
24 config = builder.create_builder_config()
25
26 # NEW API: memory pool limit
27 config.set_memory_pool_limit(trt.MemoryPoolType.WORKSPACE, 1 << 30) # 1GB
28
29 # Enable FP16
30 if builder.platform_has_fast_fp16:
31     config.set_flag(trt.BuilderFlag.FP16)
32
```

**Çıktı: TensorRT'nin
optimize ettiği engine
dosyası**

TensorRT şu işlemleri yapıyor:
ONNX'i satır satır okuyor
Hangi layer'ların olduğunu anlıyor
Layer graph'ını iç network yapısına dönüştürüyor

```
[ ] 35
    36 if engine_serialized is None:
    37     raise RuntimeError("ENGINE BUILD FAILED")
    38
    39 print("Saving engine...")
    40 with open(engine_file, "wb") as f:
    41     f.write(engine_serialized)
    42
    43 print("DONE - Saved:", engine_file)
    44
```

▼ ... Parsing ONNX...
Building serialized engine (TRT 10.14)...
Saving engine...
DONE - Saved: MobileNet-v2.engine

ONNX modeli başarıyla TensorRT tarafından optimize edildi ve engine dosyasına dönüştürüldü.


```
1 import numpy as np
2 import tensorrt as trt
3 from cuda import cudart
4
5 TRT_LOGGER = trt.Logger(trt.Logger.INFO)
6
7 print("Loading engine...")
8 with open("MobileNet-v2.engine", "rb") as f:
9     engine_bytes = f.read()
10     (function) Runtime: Any
11 runtime = trt.Runtime(TRT_LOGGER)
12 engine = runtime.deserialize_cuda_engine(engine_bytes)
13 context = engine.create_execution_context()
14
15 input_name = engine.get_tensor_name(0)
16 output_name = engine.get_tensor_name(1)
17
18 input_shape = engine.get_tensor_shape(input_name)
19 output_shape = engine.get_tensor_shape(output_name)
20
21 print("Input shape:", input_shape)
22 print("Output shape:", output_shape)
23
24 # Allocate GPU memory
25 input_nbytes = np.prod(input_shape) * np.float32().nbytes
26 output_nbytes = np.prod(output_shape) * np.float32().nbytes
27
28 _, d_input = cudart.cudaMalloc(input_nbytes)
29 _, d_output = cudart.cudaMalloc(output_nbytes)
30
31 # Dummy input
32 input_data = np.random.randn(*input_shape).astype(np.float32)
33
```

1. Engine dosyasını yüklüyor
2. TensorRT runtime oluşturuyor
3. GPU tarafında model çalıştırma bağlamı yaratıyor
4. Input ve output tensör bilgilerini çıkarıyor
5. GPU belleği ayırıyor
6. Rastgele test verisi oluşturuyor


```
34 # Copy input to GPU
35 cudart.cudaMemcpy(d_input, input_data.ctypes.data, input_nbytes, cudart.cudaMemcpyKind.cudaMemcpyHostToDevice)
36
37 # Bind pointers
38 context.set_tensor_address(input_name, d_input)
39 context.set_tensor_address(output_name, d_output)
40
41 # Execute
42 cudart.cudaDeviceSynchronize()
43 context.execute_async_v3(0)
44 cudart.cudaDeviceSynchronize()
45
46 # Copy back result
47 output_data = np.empty(output_shape, dtype=np.float32)
48 cudart.cudaMemcpy(output_data.ctypes.data, d_output, output_nbytes, cudart.cudaMemcpyKind.cudaMemcpyDeviceToHost)
49
50 print("Inference OK")
51 print("Output shape:", output_data.shape)
52 print(output_data[0][:10]) # first 10 logits
53
```

Loading engine...

Input shape: (1, 3, 224, 224)

Output shape: (1, 1000)

Inference OK

Output shape: (1, 1000)

[-0.36376953	4.34375	-0.1887207	-0.49121094	1.3623047	-1.3818359
0.09008789	1.6152344	-0.1328125	-0.00775146]		

Kullanılan Metrikler

Batch Latency: Bütün batch'in tamamını işlemek için geçen süre

Per-Image Latency: Tek bir örneğin ortalama işlenme süresi

FPS (Throughput): Saniyede işlenen örnek sayısı

FP16 VS FP32

=== FINAL RESULTS ===

Engine	Latency (ms)	Throughput (FPS)
FP32	0.4295	2328.08
FP16	0.3039	3290.60

FP16 motoru, FP32'ye kıyasla hem daha düşük latency sunuyor hem de daha yüksek throughput üretiyor.

- FP16, FP32'nin yarısı kadar bit kullanır. Bu nedenle bellekten GPU'ya ve GPU içinde çok daha az veri taşınır. Bu, bant genişliği kullanımını düşürür ve hesaplamayı hızlandırır.
- NVIDIA GPU'lar, FP16 işlemleri için özel tasarlanmış Tensor Core donanımına sahiptir. Bu birimler FP16 matris çarpımlarını FP32'ye göre 4 ila 8 kat daha hızlı işleyebilir.
- FP16 daha küçük olduğundan aynı cache alanına daha fazla veri sığar. Bu da kernel içi bellek erişimlerini hızlandırır.

Batch sizes 1, 2, 4, 8, 16, 32, 64

Batch size 1'den 64'e kadar bütün ölçümlerde FP16, FP32'den anlamlı şekilde daha düşük latency üretmiştir.

==== FP32 Benchmarks =====

Batch 1:	Batch Lat = 0.475 ms,	Per-img = 0.4752 ms,	FPS = 2104.5
Batch 2:	Batch Lat = 0.366 ms,	Per-img = 0.1829 ms,	FPS = 5468.3
Batch 4:	Batch Lat = 0.365 ms,	Per-img = 0.0913 ms,	FPS = 10947.9
Batch 8:	Batch Lat = 0.368 ms,	Per-img = 0.0460 ms,	FPS = 21730.3
Batch 16:	Batch Lat = 0.366 ms,	Per-img = 0.0229 ms,	FPS = 43762.1
Batch 32:	Batch Lat = 0.366 ms,	Per-img = 0.0114 ms,	FPS = 87515.6
Batch 64:	Batch Lat = 0.370 ms,	Per-img = 0.0058 ms,	FPS = 172975.1

==== FP16 Benchmarks =====

Batch 1:	Batch Lat = 0.297 ms,	Per-img = 0.2965 ms,	FPS = 3372.1
Batch 2:	Batch Lat = 0.294 ms,	Per-img = 0.1469 ms,	FPS = 6806.8
Batch 4:	Batch Lat = 0.294 ms,	Per-img = 0.0735 ms,	FPS = 13606.4
Batch 8:	Batch Lat = 0.294 ms,	Per-img = 0.0367 ms,	FPS = 27220.0
Batch 16:	Batch Lat = 0.294 ms,	Per-img = 0.0184 ms,	FPS = 54384.4
Batch 32:	Batch Lat = 0.294 ms,	Per-img = 0.0092 ms,	FPS = 108767.9
Batch 64:	Batch Lat = 0.294 ms,	Per-img = 0.0046 ms,	FPS = 217834.6

Batch sizes 1, 2, 4, 8, 16, 32, 64

==== FP32 Benchmarks =====

Batch 1:	Batch Lat = 0.475 ms,	Per-img = 0.4752 ms,	FPS = 2104.5
Batch 2:	Batch Lat = 0.366 ms,	Per-img = 0.1829 ms,	FPS = 5468.3
Batch 4:	Batch Lat = 0.365 ms,	Per-img = 0.0913 ms,	FPS = 10947.9
Batch 8:	Batch Lat = 0.368 ms,	Per-img = 0.0460 ms,	FPS = 21730.3
Batch 16:	Batch Lat = 0.366 ms,	Per-img = 0.0229 ms,	FPS = 43762.1
Batch 32:	Batch Lat = 0.366 ms,	Per-img = 0.0114 ms,	FPS = 87515.6
Batch 64:	Batch Lat = 0.370 ms,	Per-img = 0.0058 ms,	FPS = 172975.1

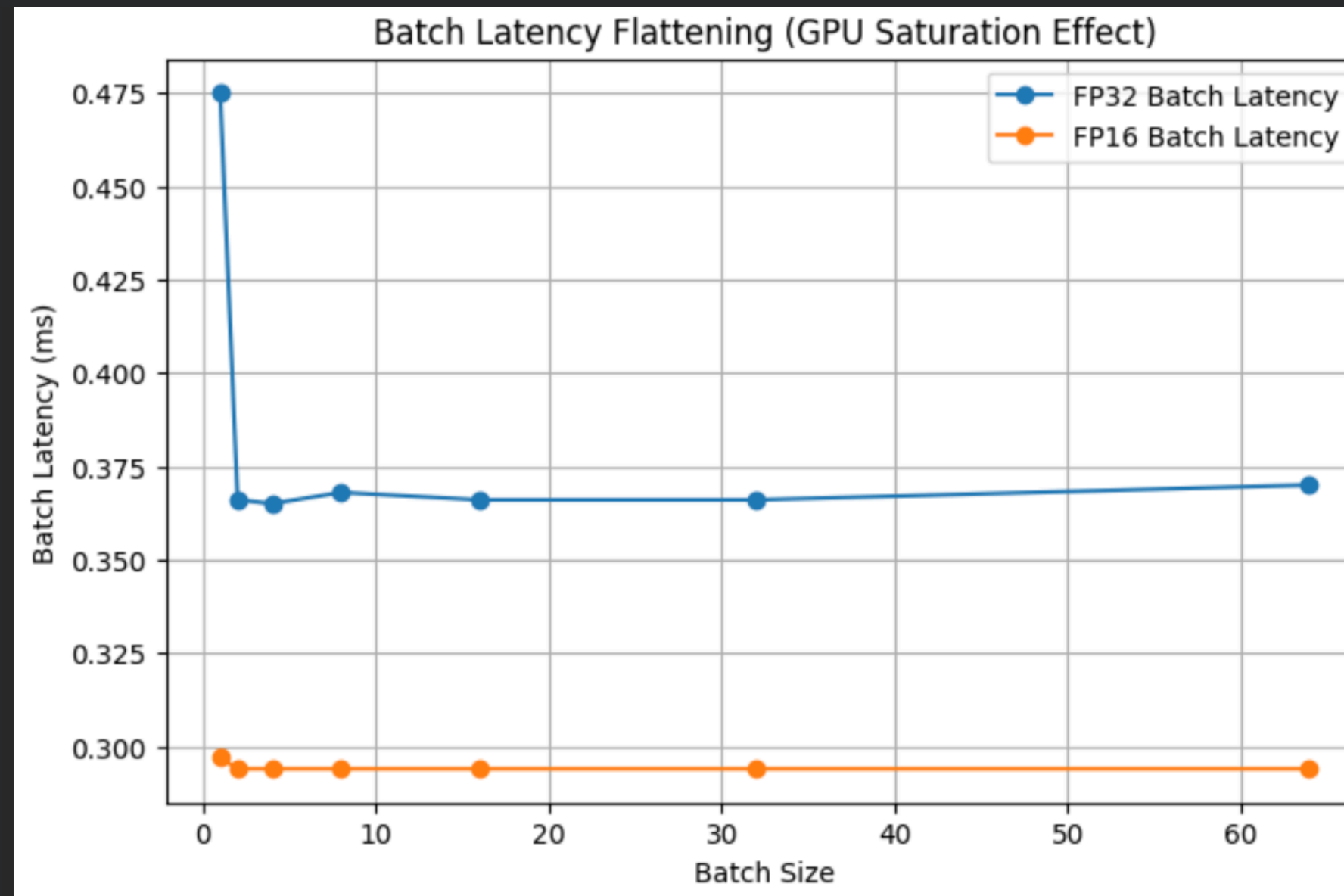
==== FP16 Benchmarks =====

Batch 1:	Batch Lat = 0.297 ms,	Per-img = 0.2965 ms,	FPS = 3372.1
Batch 2:	Batch Lat = 0.294 ms,	Per-img = 0.1469 ms,	FPS = 6806.8
Batch 4:	Batch Lat = 0.294 ms,	Per-img = 0.0735 ms,	FPS = 13606.4
Batch 8:	Batch Lat = 0.294 ms,	Per-img = 0.0367 ms,	FPS = 27220.0
Batch 16:	Batch Lat = 0.294 ms,	Per-img = 0.0184 ms,	FPS = 54384.4
Batch 32:	Batch Lat = 0.294 ms,	Per-img = 0.0092 ms,	FPS = 108767.9
Batch 64:	Batch Lat = 0.294 ms,	Per-img = 0.0046 ms,	FPS = 217834.6

Hem FP32 hem FP16 için batch latency, batch 4'ten sonra neredeyse düz bir çizgiye oturuyor.

GPU saturation effect olarak bilinir:

- GPU pipeline tamamen doluyor
 - Daha fazla görüntü eklemek, ek bekleme süresi yaratmıyor
 - Hesaplama paralel şekilde GPU içinde dağılabiliyor
- Bu nedenle batch 8 ile batch 64 arasında latency neredeyse aynıdır.



Batch size 64, batch size 1'e göre 64× değil → 64× üzerinde
throughput sağlar.

Nedeni, GPU'nun daha verimli şekilde doldurulmasıdır.

==== FP32 Benchmarks =====

Batch 1:	Batch Lat = 0.475 ms,	Per-img = 0.4752 ms,	FPS = 2104.5
Batch 2:	Batch Lat = 0.366 ms,	Per-img = 0.1829 ms,	FPS = 5468.3
Batch 4:	Batch Lat = 0.365 ms,	Per-img = 0.0913 ms,	FPS = 10947.9
Batch 8:	Batch Lat = 0.368 ms,	Per-img = 0.0460 ms,	FPS = 21730.3
Batch 16:	Batch Lat = 0.366 ms,	Per-img = 0.0229 ms,	FPS = 43762.1
Batch 32:	Batch Lat = 0.366 ms,	Per-img = 0.0114 ms,	FPS = 87515.6
Batch 64:	Batch Lat = 0.370 ms,	Per-img = 0.0058 ms,	FPS = 172975.1

==== FP16 Benchmarks =====

Batch 1:	Batch Lat = 0.297 ms,	Per-img = 0.2965 ms,	FPS = 3372.1
Batch 2:	Batch Lat = 0.294 ms,	Per-img = 0.1469 ms,	FPS = 6806.8
Batch 4:	Batch Lat = 0.294 ms,	Per-img = 0.0735 ms,	FPS = 13606.4
Batch 8:	Batch Lat = 0.294 ms,	Per-img = 0.0367 ms,	FPS = 27220.0
Batch 16:	Batch Lat = 0.294 ms,	Per-img = 0.0184 ms,	FPS = 54384.4
Batch 32:	Batch Lat = 0.294 ms,	Per-img = 0.0092 ms,	FPS = 108767.9
Batch 64:	Batch Lat = 0.294 ms,	Per-img = 0.0046 ms,	FPS = 217834.6

Per-image latency batch size arttıkça çok düşüyor

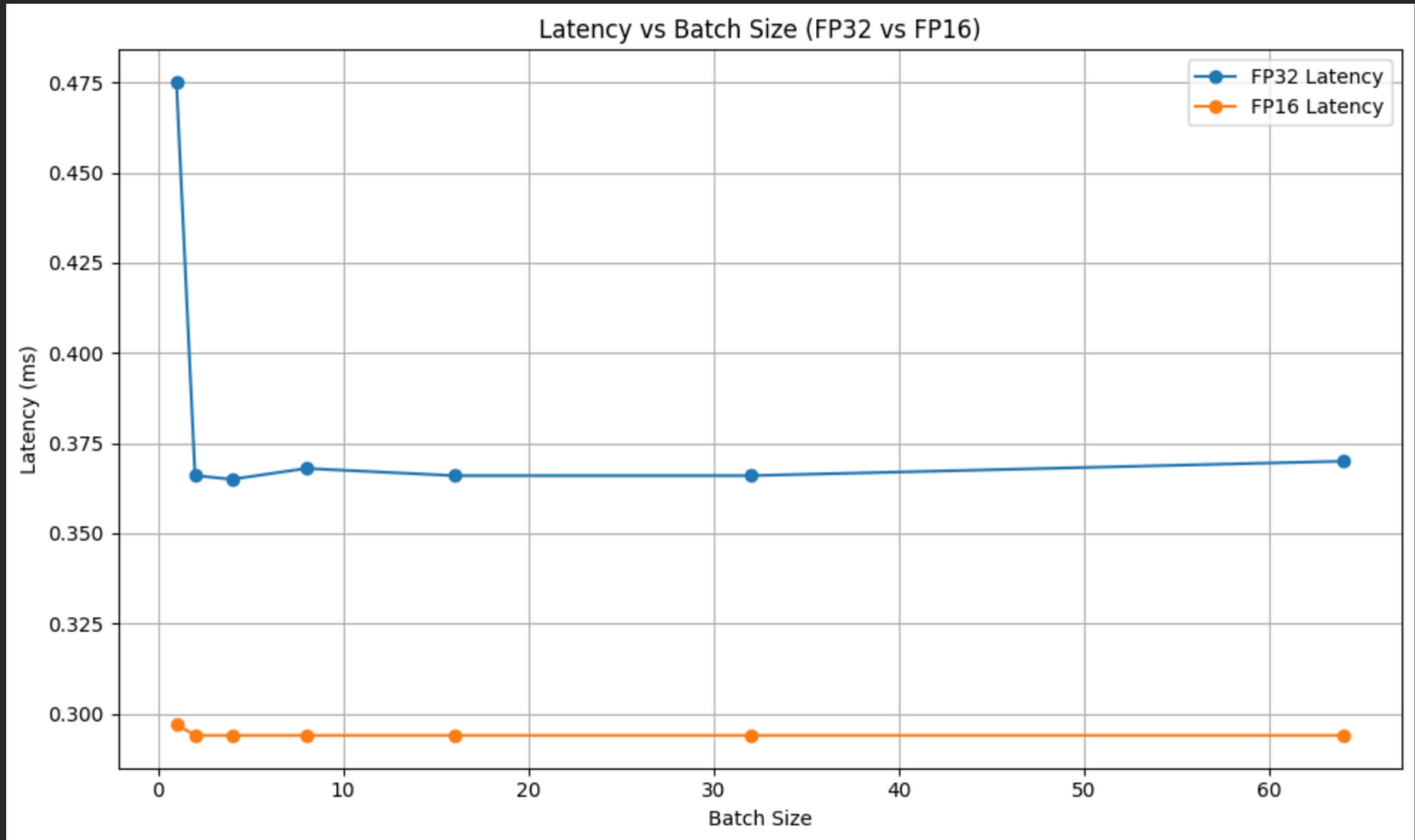
Bu da GPU'nun mini-batch optimizasyonları sayesinde gerçekleşir. GPU aynı anda çok sayıda görüntüyü işleyebilir, bu da tek görüntüye düşen maliyeti azaltır.

==== FP32 Benchmarks =====

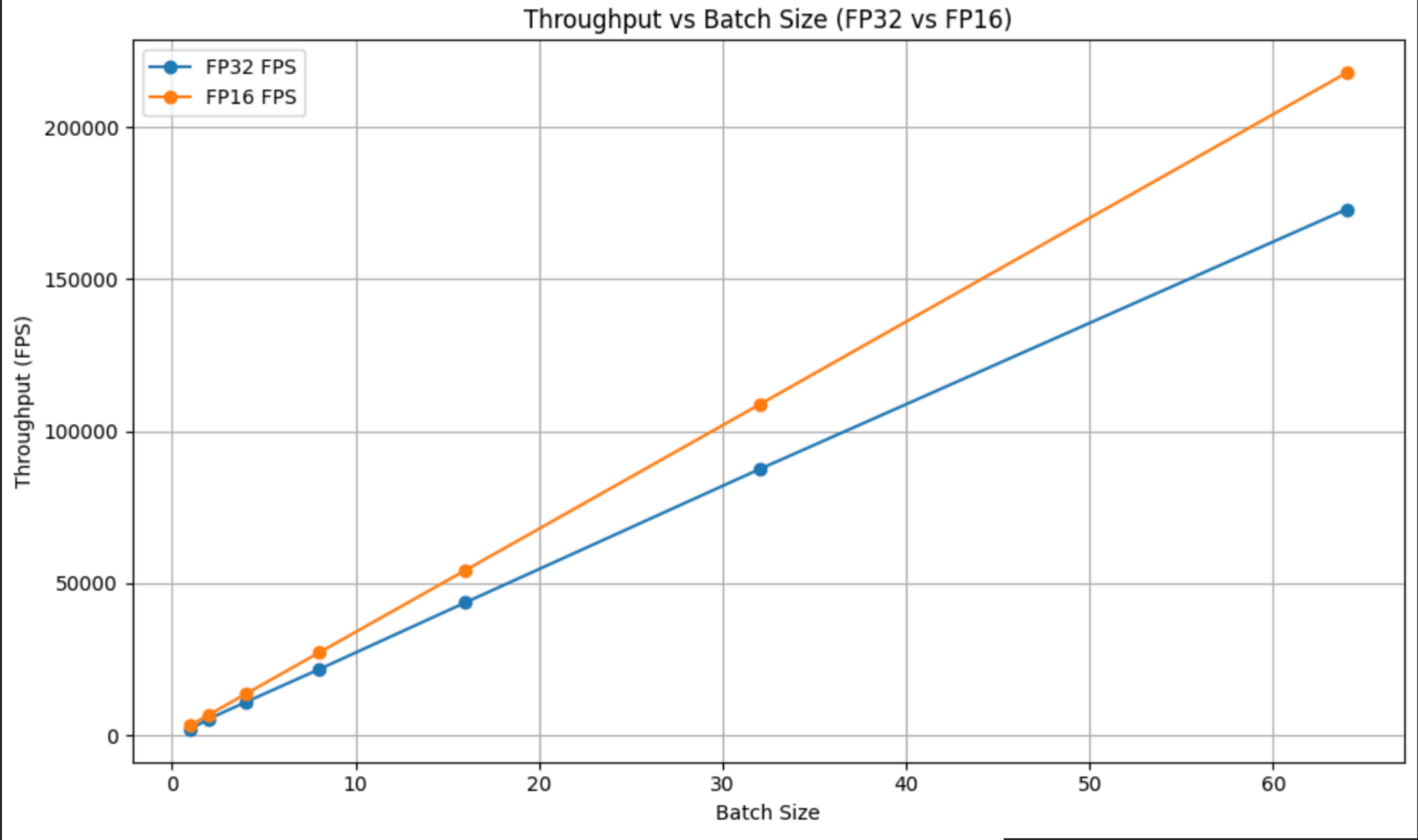
Batch 1:	Batch Lat = 0.475 ms,	Per-img = 0.4752 ms,	FPS = 2104.5
Batch 2:	Batch Lat = 0.366 ms,	Per-img = 0.1829 ms,	FPS = 5468.3
Batch 4:	Batch Lat = 0.365 ms,	Per-img = 0.0913 ms,	FPS = 10947.9
Batch 8:	Batch Lat = 0.368 ms,	Per-img = 0.0460 ms,	FPS = 21730.3
Batch 16:	Batch Lat = 0.366 ms,	Per-img = 0.0229 ms,	FPS = 43762.1
Batch 32:	Batch Lat = 0.366 ms,	Per-img = 0.0114 ms,	FPS = 87515.6
Batch 64:	Batch Lat = 0.370 ms,	Per-img = 0.0058 ms,	FPS = 172975.1

==== FP16 Benchmarks =====

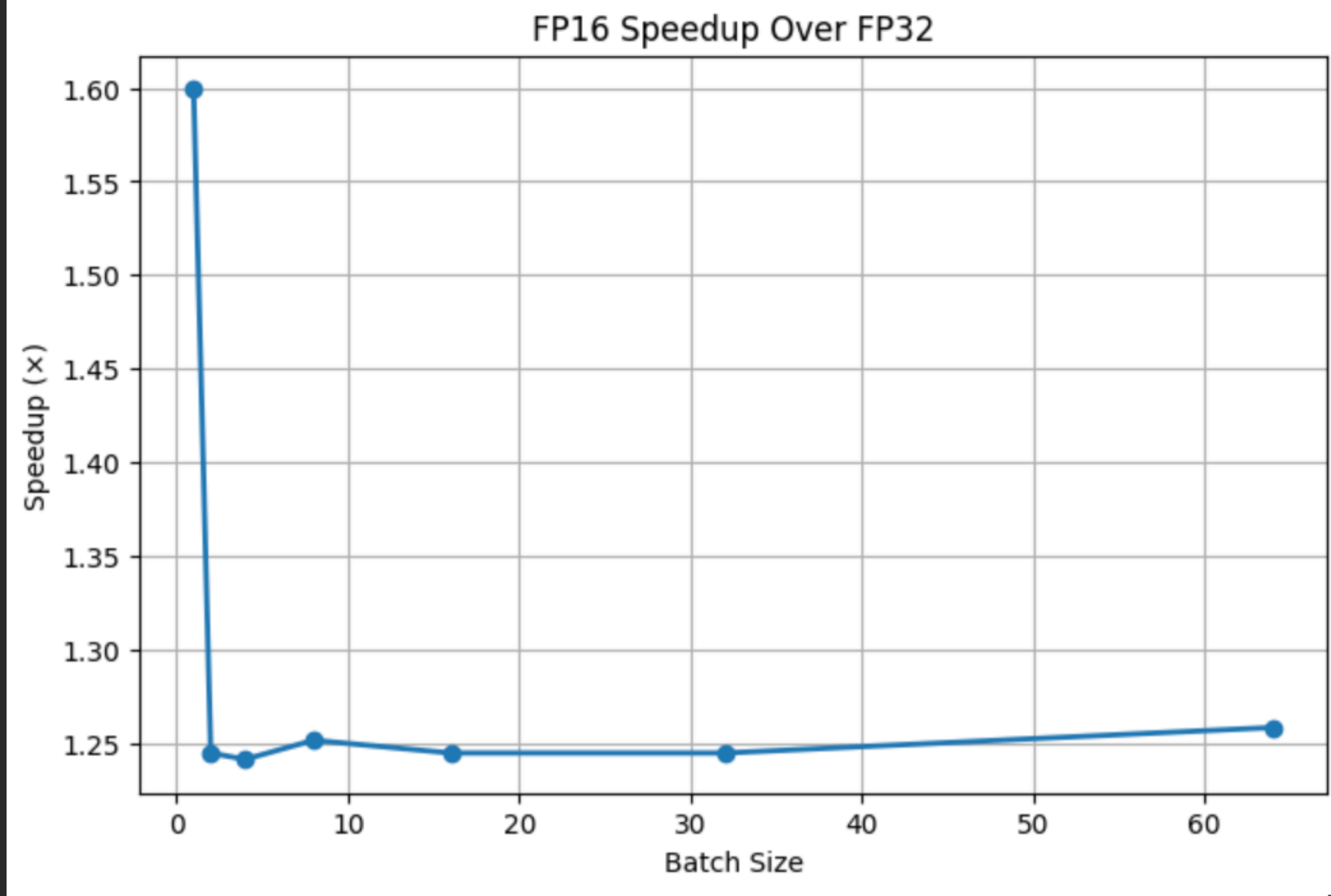
Batch 1:	Batch Lat = 0.297 ms,	Per-img = 0.2965 ms,	FPS = 3372.1
Batch 2:	Batch Lat = 0.294 ms,	Per-img = 0.1469 ms,	FPS = 6806.8
Batch 4:	Batch Lat = 0.294 ms,	Per-img = 0.0735 ms,	FPS = 13606.4
Batch 8:	Batch Lat = 0.294 ms,	Per-img = 0.0367 ms,	FPS = 27220.0
Batch 16:	Batch Lat = 0.294 ms,	Per-img = 0.0184 ms,	FPS = 54384.4
Batch 32:	Batch Lat = 0.294 ms,	Per-img = 0.0092 ms,	FPS = 108767.9
Batch 64:	Batch Lat = 0.294 ms,	Per-img = 0.0046 ms,	FPS = 217834.6



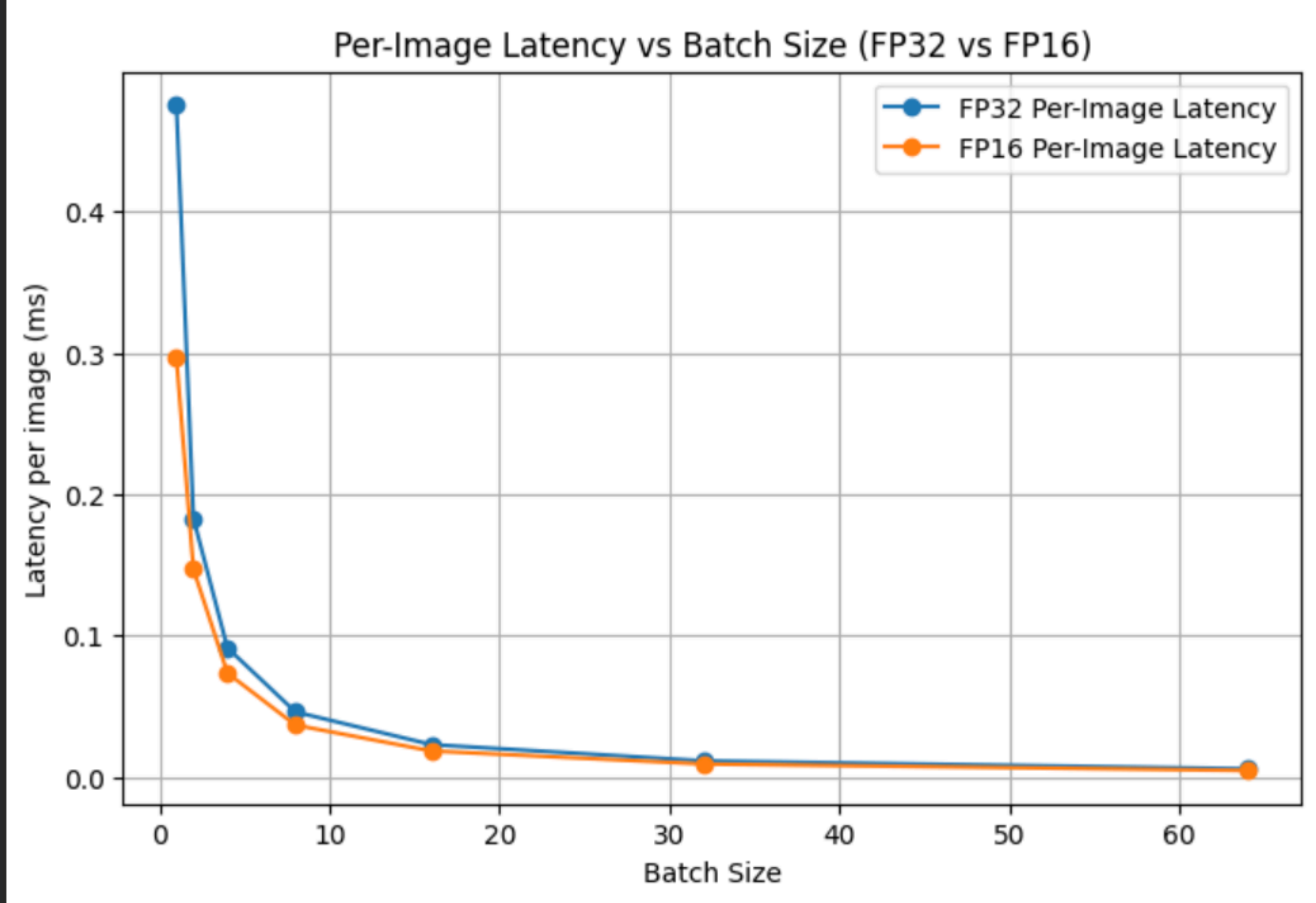
batch size arttıkça hem FP32 hem FP16 için batch latency'nin hızlıca sabit bir değere oturduğunu ve FP16'nın tüm batch size'larda FP32'den belirgin şekilde daha düşük gecikme sunduğunu görüyoruz.



Bu grafik, batch size arttıkça throughput'un doğrusal biçimde yükseldiğini ve FP16'nın her batch size'da FP32'ye göre daha yüksek FPS sağlayarak çok daha verimli çalıştığını gösterir.



Bu grafik, FP16'in FP32'ye göre tüm batch size değerlerinde yaklaşık 1.25x hızlanma sağladığını ve özellikle küçük batch size'da daha belirgin avantaj sunduğunu gösterir.



Bu grafik, batch size büyüdükçe hem FP32 hem FP16 için görüntü başına gecikmenin hızla azaldığını ve FP16'nın her noktada FP32'den biraz daha düşük gecikme verdiğini gösterir.

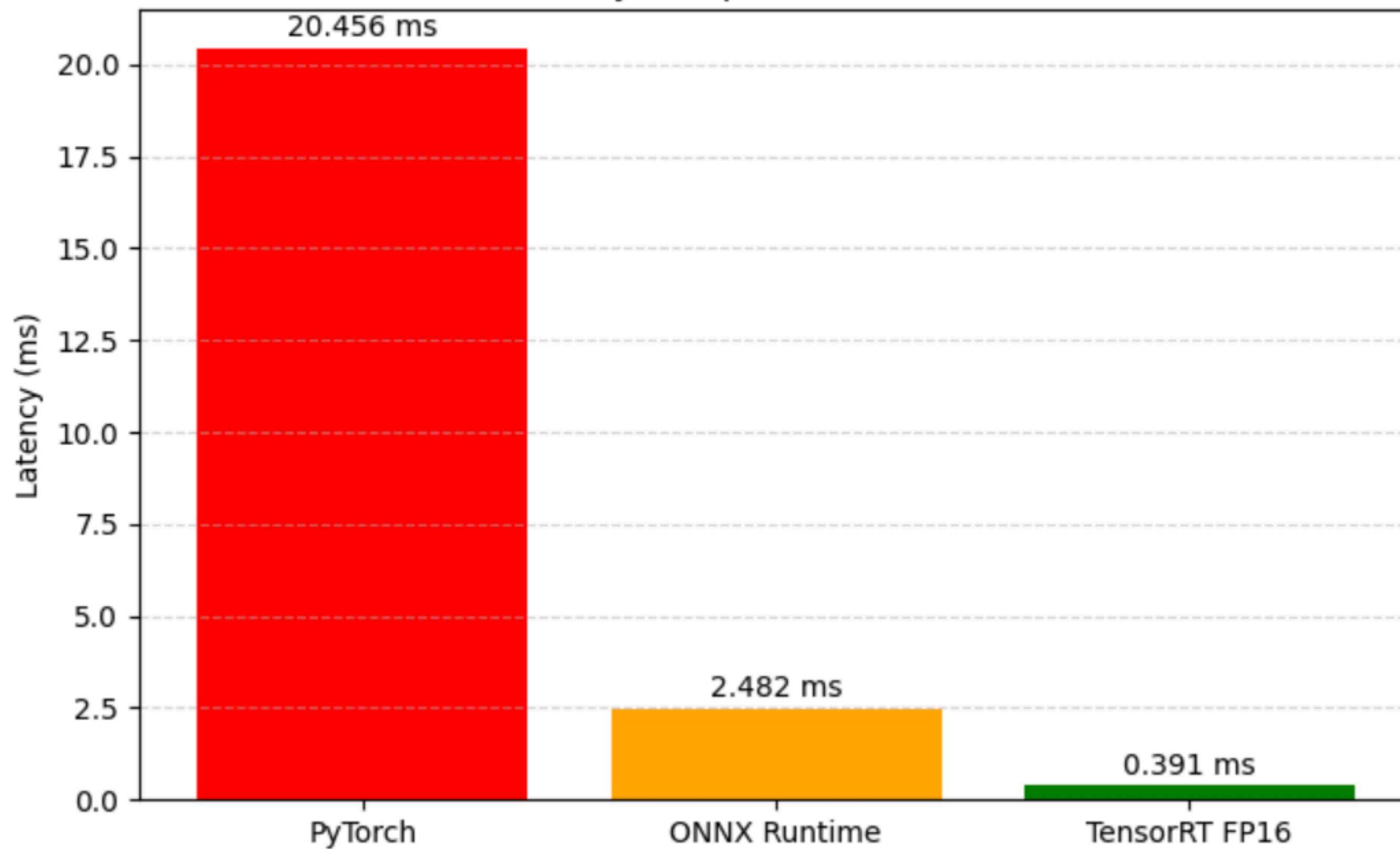
Pytorch vs ONNX vs TensorRT

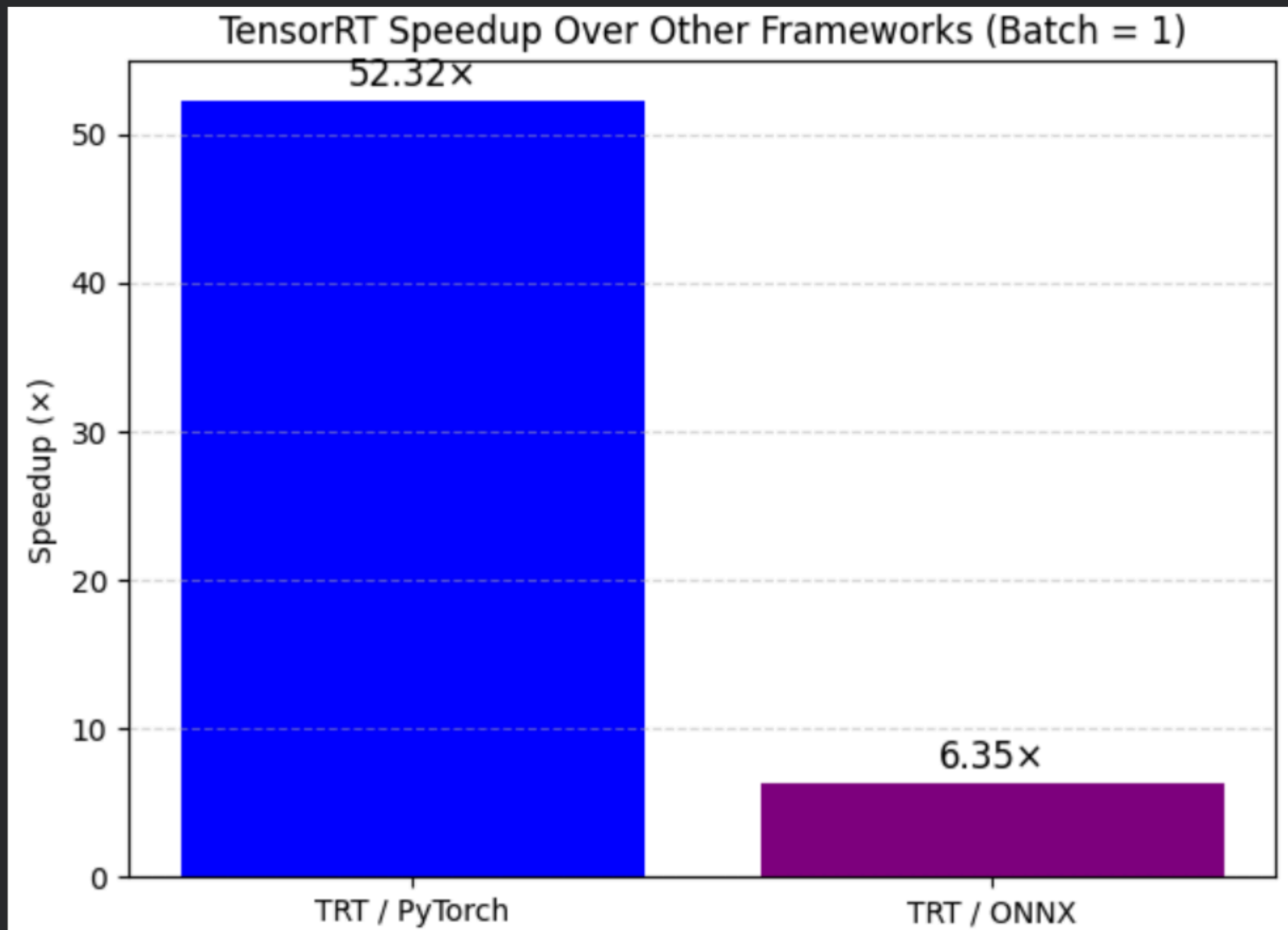
```
PyTorch Latency: 27.914714336395264 ms  
PyTorch FPS: 35.82340080393364  
ONNX Runtime Latency: 2.971470355987549 ms  
ONNX Runtime FPS: 336.5337291637414  
TensorRT FP16 Latency: 0.38864803314208984 ms  
TensorRT FP16 FPS: 2573.022155587237
```

TensorRT FP16, 0.38 ms gecikme ve 2573 FPS ile en hızlı çözüm olurken, ONNX Runtime yaklaşık 3 ms gecikme ve 336 FPS ile orta seviyede kalıyor, PyTorch ise 27 ms gecikme ve 35 FPS civarında en yavaş sonuçları veriyor.

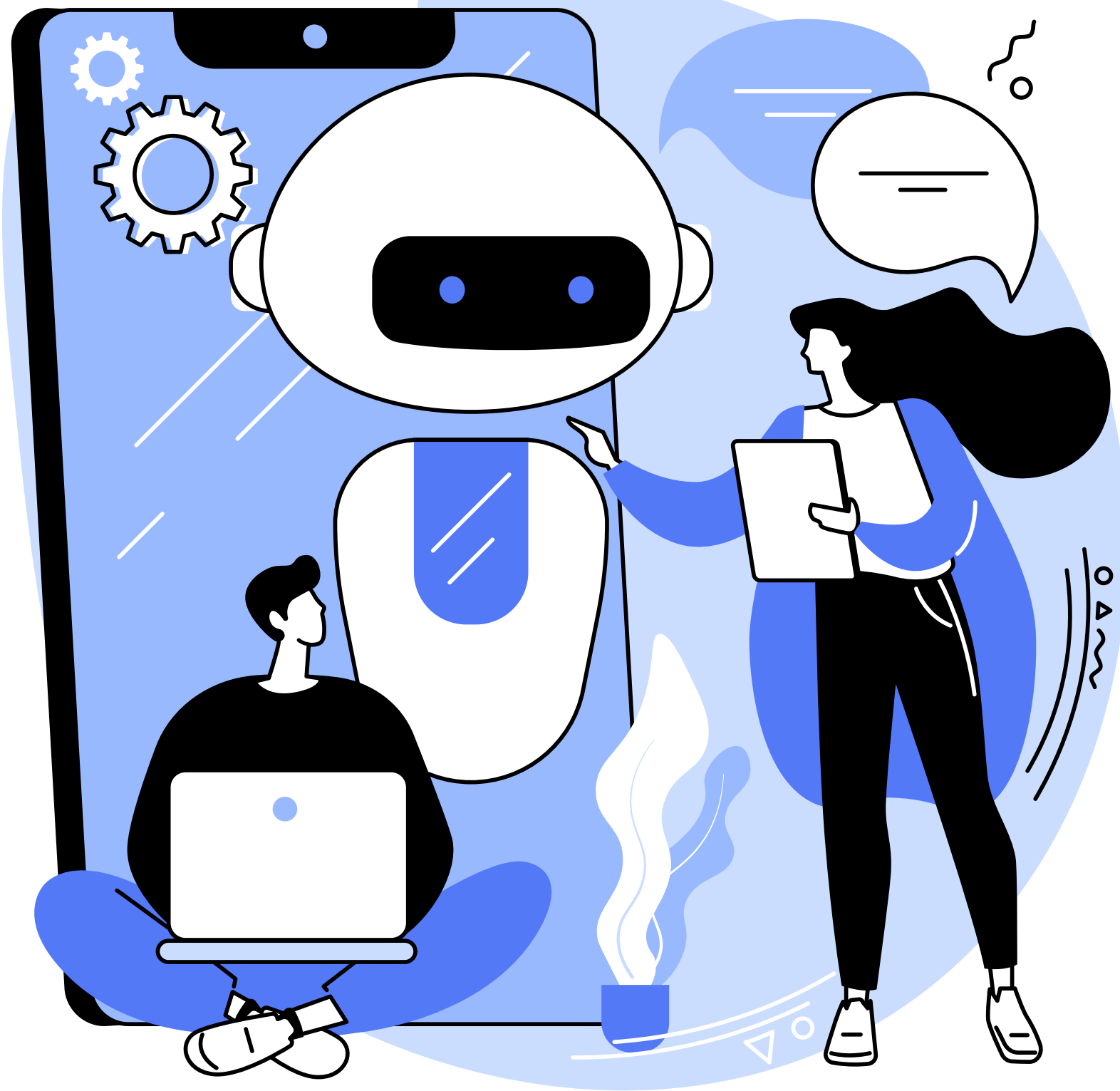
Yani aynı modeli aynı girdilerle çalıştırdığımızda TensorRT FP16, PyTorch'tan yaklaşık 70 kat, ONNX Runtime'dan ise yaklaşık 8 kat daha yüksek throughput sağlıyor.

Latency Comparison (Batch = 1)





$$\text{Speedup} = \frac{\text{Baseline Latency}}{\text{Optimized Latency}}$$



MobileNet Nsight Systems Profiling

[7]

✓ 4s

```
1 !sudo dpkg -i NsightSystems-linux-cli-public-2025.6.1.190-3689520.deb
2
```

✓

```
Selecting previously unselected package nsight-systems-cli-2025.6.1.
(Reading database ... 121713 files and directories currently installed.)
Preparing to unpack NsightSystems-linux-cli-public-2025.6.1.190-3689520.deb ...
Unpacking nsight-systems-cli-2025.6.1 (2025.6.1.190-256136895201v0) ...
Setting up nsight-systems-cli-2025.6.1 (2025.6.1.190-256136895201v0) ...
update-alternatives: using /opt/nvidia/nsight-systems-cli/2025.6.1/target-linux-x64/nsys to provide /usr/local/bin/nsys (nsys) in auto mode
```

[9]

✓ 0s

```
1 !nsys --version
2
```

✓

```
NVIDIA Nsight Systems version 2025.6.1.190-256136895201v0
```




```
1 %%writefile trt_inference.py
2 from cuda import cudart
3 import numpy as np
4 import time
5 import cv2
6 import tensorrt as trt
7 import os
8
9 engine_path = "MobileNet-v2.engine"
10
11 TRT_LOGGER = trt.Logger()
12 runtime = trt.Runtime(TRT_LOGGER)
13
14 with open(engine_path, "rb") as f:
15     engine = runtime.deserialize_cuda_engine(f.read())
16
17 context = engine.create_execution_context()
18
19 input_name = [engine.get_tensor_name(i)
20               for i in range(engine.num_io_tensors)
21               if engine.get_tensor_mode(engine.get_tensor_name(i)) == trt.TensorIOMode.INPUT][0]
22
23 output_name = [engine.get_tensor_name(i)
24                for i in range(engine.num_io_tensors)
25                if engine.get_tensor_mode(engine.get_tensor_name(i)) == trt.TensorIOMode.OUTPUT][0]
26
27 output_shape = tuple(engine.get_tensor_shape(output_name))
28
29 # Allocate memory
30 input_nbytes = 1 * 3 * 224 * 224 * 4
31 output_nbytes = np.prod(output_shape) * 4
32
33 _, d_input = cudart.cudaMalloc(input_nbytes)
34 _, d_output = cudart.cudaMalloc(output_nbytes)
35
```

Bu kısım MobileNet v2 için oluşturulmuş TensorRT engine dosyasını yükleyip çalıştırma bağlamı oluşturuyor. Ardından giriş ve çıkış tensörlerinin adlarını ve boyutlarını alıyor, GPU belleğinde giriş ve çıkış için gerekli alanları ayırıyor. Bu hazırlık adımları modelin TensorRT üzerinden hızlı inference yapabilmesi için gerekli başlangıç kurulumudur.


```
35
36 # Dummy input
37 x = np.random.rand(1,3,224,224).astype(np.float32)
38
39 # Warm up
40 for _ in range(5):
41     cudart.cudaMemcpy(d_input, x.ctypes.data, input_nbytes,
42                       cudart.cudaMemcpyKind.cudaMemcpyHostToDevice)
43     context.set_tensor_address(input_name, d_input)
44     context.set_tensor_address(output_name, d_output)
45     context.execute_async_v3(0)
46
```

... Writing trt_inference.py

Modeli test etmek için rastgele bir giriş oluşturduk.

Modeli gerçek zamanlı kullanım öncesinde ısındırmak için birkaç kez çalıştırıyoruz.

Önemli adımlar:

- CPU üzerinde oluşturduğumuz dummy input verisini GPU belleğine kopyalıyoruz.
- Giriş ve çıkış tensörlerinin GPU adreslerini context içine bağlıyoruz.
- `execute_async_v3` ile modeli çalıştırıyoruz. Bu çağrı GPU üzerinde async çalışıyor.

!nsys profile -o mobilenet_report python3 trt_inference.py

python3 trt_inference.py

Python kodunu Nsight Systems ile çalıştırıyoruz

nsys profile

Nsight Systems “profiler” devreye giriyor

- GPU kernel çağrılarını
- CUDA fonksiyonlarını
- TensorRT’nin çalıştırdığı tüm operasyonları
- Memory kopyalama işlemlerini
- CPU thread aktivitelerini
- GPU stream kullanımını

Kaydediyor.

-o mobilenet_report

Nsight'ın tüm topladığı
veriyi şu dosyaya
kaydetmesini sağlar:

mobilenet_report.nsys-rep

**Sonra bu dosyayı
Nsight Systems
GUI ile açıyoruz.**

Kod çalışırken:

- Python'ın kendi threadleri
 - Data loader threadleri
 - TensorRT'nin backend threadleri
 - CUDA driver threadleri
- olduğu için toplam 21 thread açılmış.

Launch
python3 trt_inference.py
Modeli çalıştırdığımız Python programı.
Start
Profiling CLI tarafından başlatıldı.
Process exit
Kod başarıyla (exit code 0) tamamlandı.
Stop
Profiling, süreç tamamlanınca otomatik durdu

Bu modelin TensorRT üzerinden çalışmasının toplam profillendiği süre yaklaşık 1.08 saniye.

Nsight Systems, aşağıdaki olayları toplar:

- CUDA kernel launch
- Memory copy
- TensorRT kernel çağrılar
- Python thread aktiviteleri
- Stream senkronizasyonları
- Kütüphane fonksiyon çağrılar

Toplam 7 bin 751 olay var. Bu, modelin GPU üzerinde aktif bir şekilde çalıştığını gösteren güçlü bir veri.

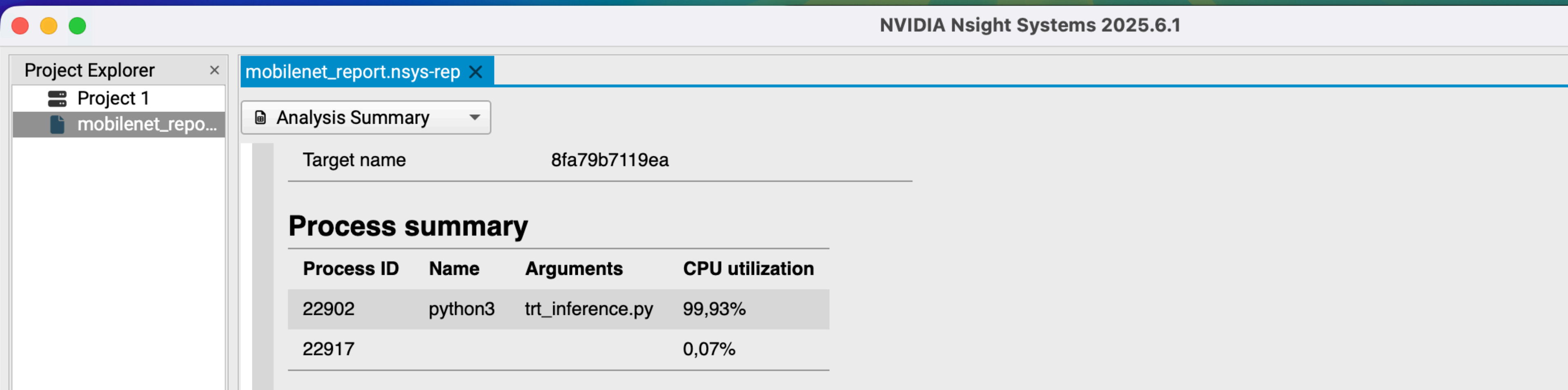
8fa79b7119ea (0:0)

Target

Hostname	8fa79b7119ea
Local time at t=0	2025-12-08T17:08:23.032
UTC time at t=0	2025-12-08T14:08:23.032Z
TSC value at t=0	5428860857574
Platform	Linux
OS	Ubuntu 22.04.4 LTS
Hardware platform	x86_64
Serial number	Local (CLI)
CPU description	Intel(R) Xeon(R) CPU @ 2.20GHz
GPU descriptions	NVIDIA A100-SXM4-40GB
NVIDIA driver version	550.54.15
Max EMC frequency	1,60 GHz
CPU context switch	supported
GPU context switch	supported
Guest VM id	0
Tunnel traffic through SSH	no
Timestamp counter	supported
Linux paranoid level	2
System UID	pid:[4026532689]
Profiling session UUID	5e1dc84d-8af9-4611-a798-729e00d2dc1c
Target name	8fa79b7119ea

Makinedeki işlemci:
Intel Xeon CPU @ 2.20 GHz
Bu CPU inference sırasında
datayı GPU'ya hazırlamak gibi
görevlerde rol aldı.

Profilin alındığı GPU:
NVIDIA A100-SXM4-40GB
Bu bizim tüm TensorRT
kernel'larını çalıştıran güçlü
datacenter GPU'su.



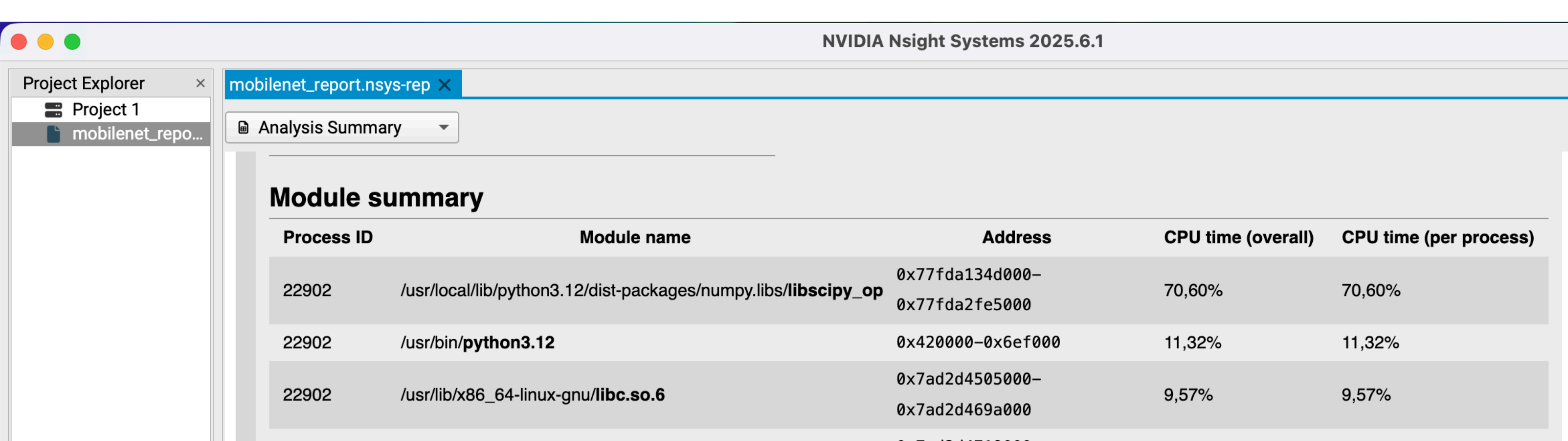
Process ID: 22902

Bu, bizim TensorRT kodumuzu çalıştıran Python işlemidir. Yani şu komutla başlattığımız program:

python3 trt_inference.py

CPU Utilization: %99.93

TensorRT inference sırasında CPU yükü neredeyse tamamen python3 işleminde. Yani CPU tarafı yoğun şekilde preprocess, veri okuma, H2D transfer hazırlığı gibi işleri yapıyor. GPU ise saf compute kısmını çalıştırıyor. Bu değer CPU'nun neredeyse tamamen bu iş için çalıştığını gösteriyor.



1. Profil sonucunda CPU'nun yüzde 70'ten fazlasının preprocess aşamasında harcandığını gördük. Bu, GPU tarafı çok hızlı hesaplama yapsa bile CPU preprocessing işlemleri yüzünden bottleneck yaşandığını gösteriyor. %70,60
2. Bu Python runtime'ın kendisi: döngüleri çalıştırma, fonksiyon çağrıları, TensorRT API çağrıları, dosya yönetimi gibi işleri yapıyor. %11,32
3. Bu, Linux'un C standart kütüphanesi: dosya açma, memory allocation, string işlemleri, I/O gibi temel sistem çağrılarını yapıyor. %9,57

Bu 3'ü 91.49% yapıyor.

GPU kernel'ları çok hızlı, ancak CPU tarafı veri hazırlamak için çok fazla zaman harcıyor.

		0x7ad2d469a000		
22902	/usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2	0x7ad2d4719000– 0x7ad2d4743000	1,98%	1,98%
22902	/opt/nvidia/nsight-systems-cli/2025.6.1/target-linux-x64/libTool:	0x77fdabbd5000– 0x77fdacb27000	1,52%	1,52%
22902	/usr/local/lib/python3.12/dist-packages/tensorrt_libs/libnvinfer.	0x77fd12c81000– 0x77fd2a360000	1,52%	1,52%
22902	/usr/lib64-nvidia/libcuda.so.550.54.15	0x77fd100db000– 0x77fd10573000	1,17%	1,17%
22902	/opt/nvidia/nsight-systems-cli/2025.6.1/target-linux-x64/libTool:	0x77fdab7b8000– 0x77fdab833000	0,82%	0,82%
22902	/opt/nvidia/nsight-systems-cli/2025.6.1/target-linux-x64/libnvpe	0x77fdace46000– 0x77fdad3e7000	0,58%	0,58%
22902	/opt/nvidia/nsight-systems-cli/2025.6.1/target-linux-x64/libnvpe	0x77fdb6340000– 0x77fdb6733000	0,23%	0,23%
22902	/usr/local/lib/python3.12/dist-packages/numpy/_core/_multiarr:	0x77fda33f5000– 0x77fda3b85000	0,23%	0,23%
22902	/usr/local/lib/python3.12/dist-packages/tensorrt_bindings/tenso	0x77fd42c49000– 0x77fd42d42000	0,12%	0,12%
22902	/usr/lib/x86_64-linux-gnu/libcrypto.so.3	0x77fbc9c6e000– 0x77fbc9ecd000	0,12%	0,12%
22902	/opt/nvidia/nsight-systems-cli/2025.6.1/target-linux-x64/libcupt	0x77fbd1067000– 0x77fbd1470000	0,12%	0,12%
22902	/usr/local/lib/python3.12/dist-packages/numpy/random/_mt199:	0x77fd44ce3000– 0x77fd44cf7000	0,12%	0,12%

**TensorRT CPU
tarafında çok küçük
bir yük oluşturuyor
(%1.5)**

**Bu TensorRT’nin
“lightweight CPU +
heavy GPU execute”
mimarisine uygun.**

Project Explorer



Project 1



mobilenet_repo...

mobilenet_report.nsys-rep



Analysis Summary

Thread summary

Information about 21 threads (that have been active at least once) has been captured during the profiling session.

Information about idle threads is not represented here.

Process ID	Thread ID	Name	CPU utilization
22902	22902	python3	43,95%
22902	22923	python3	5,51%
22902	22921	python3	5,51%
22902	22920	python3	5,48%
22902	22919	python3	5,47%
22902	22924	python3	5,46%
22902	22925	python3	5,34%
22902	22922	python3	5,33%
22902	22918	python3	5,16%
22902	22928	python3	5,10%
22902	22926	python3	3,45%
22902	22927	python3	3,36%
22902	22934	CUPTI worker thread	0,38%
22902	22914	[NSys]	0,29%

Information about 7 threads with CPU utilization below 0,10% has been hidden.

Ana thread CPU'nun %44'ünü tek başına kullanıyor

- dataset klasörlerini gezme
- OpenCV ile resimleri okuma (cv2.imread)
- resize, normalize, transpose
- NumPy işlemleri
- TensorRT çağrılarını yapma

Bu sonuç uygulamanın CPU tarafında paralel çalışmadığını ve preprocess aşamasının tek bir thread'e yüklendiğini gösteriyor. Bu da TensorRT'in GPU'da çok hızlı çalışmasına rağmen CPU tarafının bottleneck oluşturduğunu doğruluyor.

CPU info

CPU core	Socket	Core type	Max frequency	MPIDR	Performance / Efficient (P/E)
#0	#0	Unknown	4,00 GHz	none	none
#1	#0	Unknown	4,00 GHz	none	none
#2	#0	Unknown	4,00 GHz	none	none
#3	#0	Unknown	4,00 GHz	none	none
#4	#0	Unknown	4,00 GHz	none	none
#5	#0	Unknown	4,00 GHz	none	none
#6	#0	Unknown	4,00 GHz	none	none
#7	#0	Unknown	4,00 GHz	none	none
#8	#0	Unknown	4,00 GHz	none	none
#9	#0	Unknown	4,00 GHz	none	none
#10	#0	Unknown	4,00 GHz	none	none
#11	#0	Unknown	4,00 GHz	none	none

Google Colab kullanıcıdan donanım detaylarını gizliyor. Bu nedenle Nsight Systems:

- çekirdeğin mimarisini
- P-core/E-core tipini
- MPIDR bilgisini

göremiyor.

Uygulamanın CPU tarafı 12 çekirdek olmasına rağmen tek thread çalıştığı için çok çekirdekli yapıdan faydalanamıyor.

Project Explorer



Project 1



mobilenet_repo...

mobilenet_report.nsys-rep X



Analysis Summary



GPU info

NVIDIA A100-SXM4-40GB

Chip Name	GA100
SM Count	108
L2 Cache Size	40,00 MiB
Memory Bandwidth	1.448,39 GiB/s
Memory Size	39,56 GiB
Core Clock	1,41 GHz
Bus Location	0000:00:04.0
UUID	2631b75f-de10-42bf-103d-05e1a329e6ba
GSP firmware version	N/A
Video accelerator tracing	Supported

NIC info

No NVIDIA NICs were detected

OpenFabrics Driver

OFED (OpenFabrics Enterprise Distribution) version: OFED is not detected.

A100'de toplam 108 adet SM vardır.

Her SM içinde yüzlerce CUDA çekirdeği bulunur.

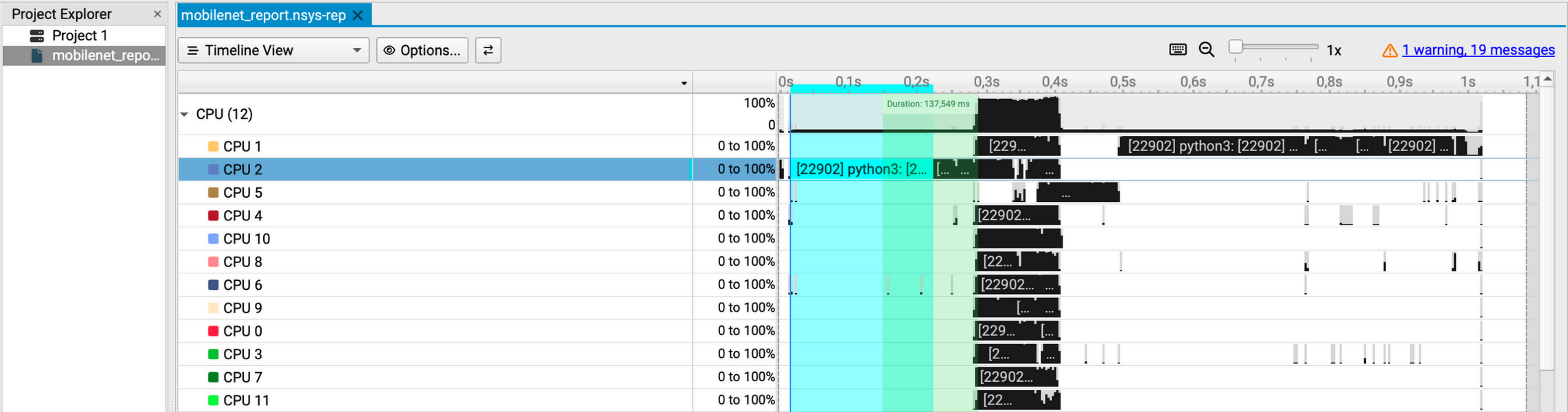
L2 cache, katmanlar arasındaki veri geçişini hızlandırır.

GPU'nun saniyede taşıyabildiği veri miktarı.

1448 GB/s çok yüksek bir bant genişliğidir. Konvolüsyon gibi yoğun bellek erişimli işlemler için kritik.

40 GB HBM2e bellek.

Büyük modelleri rahat yükleyebilir.



0.0 s – 0.3 s arası

Bu bölge preprocessing aşaması.

- input görüntüyü okuyor
- resize, normalize gibi NumPy işlemlerini uyguluyor
- Tensorrt engine için input binding hazırlıyor
- CUDA'ya aktarılacak tensörleri düzenliyor
- Python GIL yüzünden işlem tek çekirdekte sıkışıyor

Bu evrede olan şey

- CPU tek başına çalışıyor
- GPU henüz devreye girmiyor

Bu yüzden timeline'da büyük, kesintisiz bir CPU bloğu görülüyor

0.3 s – 0.5 s arası

GPU çalışmaya başlıyor, CPU ise kernel yönetimi yapıyor

Bu aşamada Tensorrt inference başlatılıyor.

CPU'nun görevleri:

- CUDA kernel çağrılarını sıraya koymak
- GPU'ya küçük görevler göndermek
- Asenkron cudaStream işlemlerini yönetmek
- Bazı kopyalama ve kontrol görevlerini yapmak
- Timeline'da neden kısa siyah bloklar var?

Bu siyah bloklar:

CPU'nun GPU'yu çalıştırırken yaptığı küçük yönetim görevlerini

- kernel launch
- senkronizasyon
- driver içi kontrol adımlarını

temsil ediyor.

0.5 s sonrası

Post processing + artan küçük CPU işleri

GPU inference bittikten sonra:

- sonuç tensörleri CPU'ya geri geliyor
- output üzerinde Python ile küçük işlemler yapılıyor
- log, print, sonuç hazırlama gibi işler yürütülüyor

Bu yüzden küçük, parçalı CPU aktiviteleri görülüyor.

Timeline neden parçalı görünüyor?

Çünkü:

Bu aşamalarda CPU kısa süreli mikro görevler çalıştırıyor

Kesintisiz bir yük yok

Python artık yoğun çalışmıyor

GEMM yükü yaklaşık %35 + %15 + %8 = %58'e yakın. - Pointwise Conv

NVIDIA Nsight Systems 2025.6.1

Depthwise convolutions ise %18 civarında

Geri kalan %20 civarı memory

hareketleri + yardımcı kernel'ler

Bu da normal bir TensorRT inference dağılımıdır.

Bu GPU'nun matris çarpımı (GEMM) kernelleri.

MobileNet mimarisinde:

depthwise conv çok hafif

fakat 1x1 convolutions çok maliyetli

Bu kernel MobileNet'in

depthwise convolution

adımlarını temsil ediyor.

Depthwise convolution

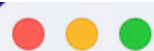
teoride daha ucuzdur:

normal conv: $M \times N \times K \times K$

depthwise conv: $M \times K \times K$

**Bu daha düşük boyutlu
feature map'lerde çalışan
GEMM'ler.**

**Bu da yine pointwise
conv işlemlerinin
başka bir setini temsil
ediyor.**



Project Explorer
Project 1
mobilenet_repo...

mobilenet_report.nsys-rep

Timeline View

Options...



1x



1 warning, 19 messages

Processes (2)

[22902] python3

CUDA HW (0000:00:04.0 - NVIDIA A100-SXM4-40GB)

[All Streams]

53,8% Kernels

35,0% sm80_xmma_gemm_f16f16_f16f16_f16_tn_n_tilsize32x32x64_stage6_warpsize2x2x1_tensor16x8x16_execute_kernel_trt

100,0% sm80_xmma_gemm_f16f16_f16f16_f16_tn_n_tilsize32x32x64_stage6_warpsize2x2x1_tensor16x8x16_execute_kernel_trt

18,3% sm50_xmma_convolution_depthwiseHMMA_FP16NHWCx8_TR3_TS3_STRIDEH1_STRIDEW1_execute_kernel_trt

100,0% sm50_xmma_convolution_depthwiseHMMA_FP16NHWCx8_TR3_TS3_STRIDEH1_STRIDEW1_execute_kernel_trt

14,7% sm80_xmma_gemm_f16f16_f16f16_f16_tn_n_tilsize64x32x64_stage7_warpsize4x1x1_tensor16x8x16_execute_kernel_trt

100,0% sm80_xmma_gemm_f16f16_f16f16_f16_tn_n_tilsize64x32x64_stage7_warpsize4x1x1_tensor16x8x16_execute_kernel_trt

7,8% sm80_xmma_gemm_f16f16_f16f16_f16_nn_n_tilsize32x32x64_stage6_warpsize2x2x1_tensor16x8x16_execute_kernel_trt

100,0% sm80_xmma_gemm_f16f16_f16f16_f16_nn_n_tilsize32x32x64_stage6_warpsize2x2x1_tensor16x8x16_execute_kernel_trt

5,9% sm50_xmma_convolution_depthwiseHMMA_FP16NHWCx8_TR3_TS3_STRIDEH2_STRIDEW2_execute_kernel_trt

3,9% sm75_xmma_fprop_implicit_gemm_indexed_wo_smem_f16f16_f16f16_f16_nhwckrsc_nhwc_tilsize128x16x32_stage1_warpsize4x1x1_g1_tensor16x8x8_alignc8_execute_kernel_trt

2,2% sm80_xmma_fprop_implicit_gemm_indexed_wo_smem_f16f16_f16f16_f16_nhwckrsc_nhwc_tilsize128x32x16_stage1_warpsize4x1x1_g1_tensor16x8x16_aligna8_alignc8_execute_kernel_trt

2,1% sm50_xmma_cublas_gemv_f16f16_f16_f16_tn_n_in_2_unit_n_launch_param16x8x8_strided_unit_stride_execute_kernel_trt

2,0% sm80_xmma_fprop_implicit_gemm_indexed_wo_smem_f16f16_f16f16_f16_nhwckrsc_nhwc_tilsize128x16x64_stage1_warpsize4x1x1_g1_tensor16x8x16_aligna8_alignc8_execute_kernel_trt

1,8% sm80_xmma_fprop_implicit_gemm_indexed_wo_smem_f16f16_f16f16_f16_nhwckrsc_nhwc_tilsize128x16x32_stage1_warpsize4x1x1_g1_tensor16x8x16_alignc8_execute_kernel_trt

1,6% copyPackedKernel

1,5% generatedNativePointwise

1,3% __myl_MaxMinMean_0xfd256d3d1fafd5798a2f23d1d8001c7c

0,9% __myl_ReshMean_0x64e4abc98c1d9dad5b84c967e2991378

0,7% __myl_Cast_0x33c744558abe44511fee3fa6338a6b82

46,2% Memory

mobilenet_report.nsys-rep X

Timeline View

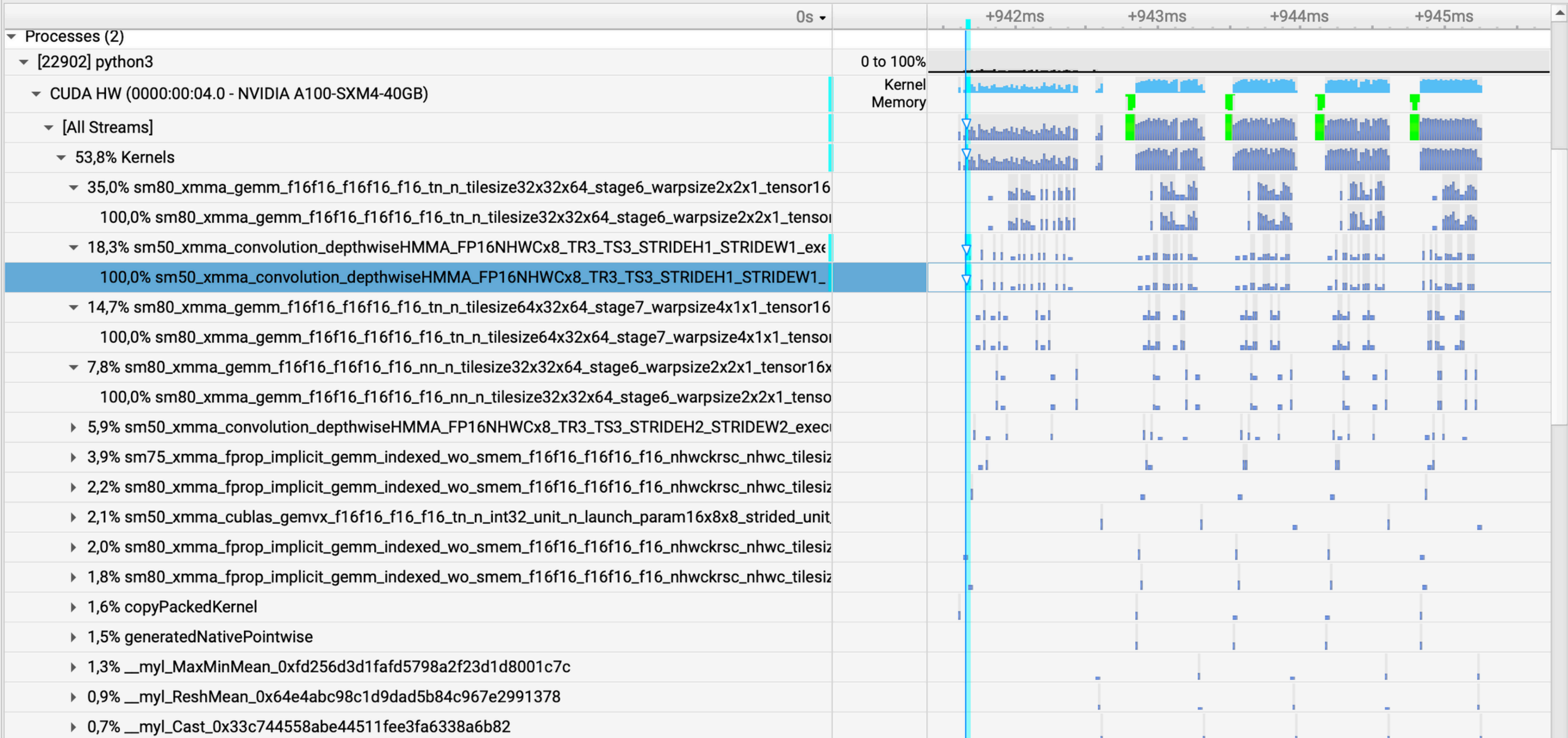
Options...

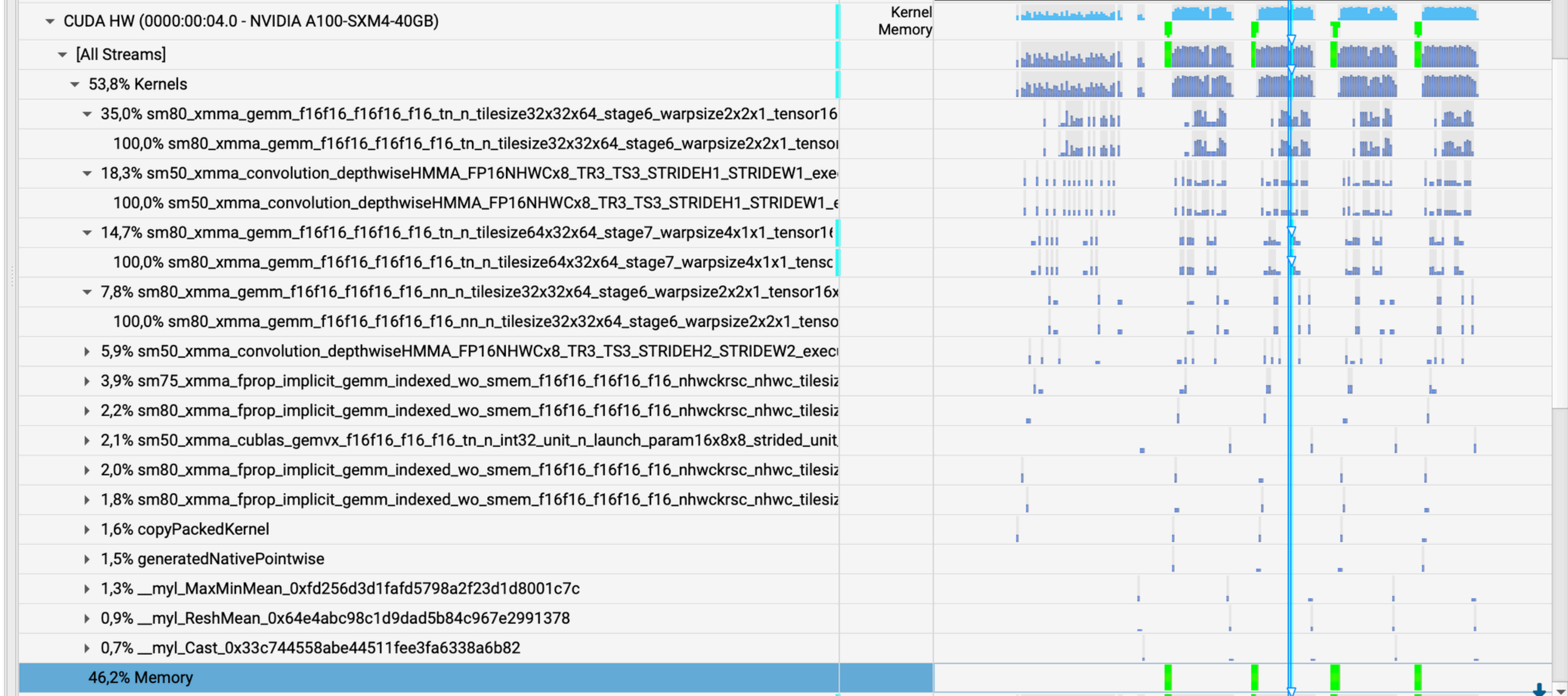


1x



1 warning, 19 messages





Mavi compute kernel’lerinin yanında memory barları çok küçük kalıyor.

Bu iyi bir şey.

- **Modelin darboğazı memory değil compute**
- **GPU’nun bant genişliği etkili kullanılıyor**
- **Katmanlar arası tensor taşınması minimal gecikme oluşturmuş**

Memory altında görünen yüzde değerleri zaman yüzdesi değil event yüzdesidir. Zaman çizelgesine baktığımızda memory event’lerinin çok kısa sürdüğünü görüyoruz ve inference’ın bottleneck’i memory değil compute kernel’leridir.

▼ 62,1% Default stream 7 **Burası modelin çoğu kernel'inin çalıştığı ana GPU stream'i.**

▼ 86,7% Kernels

▶ 35,0% sm80_xmma_gemm_f16f16_f16f16_f16_tn_n_tilesiz32x32x64_stage6_warpsiz2x2x1_tensor16

▶ 18,3% sm50_xmma_convolution_depthwiseHMMA_FP16NHWCx8_TR3_TS3_STRIDEH1_STRIDEW1_exe

▶ 14,7% sm80_xmma_gemm_f16f16_f16f16_f16_tn_n_tilesiz64x32x64_stage7_warpsiz4x1x1_tenso ↓

▶ 7,8% sm80_xmma_gemm_f16f16_f16f16_f16_nn_n_tilesiz32x32x64_stage6_warpsiz2x2x1_tensor16x

▶ 5,9% sm50_xmma_convolution_depthwiseHMMA_FP16NHWCx8_TR3_TS3_STRIDEH2_STRIDEW2_execi

10 kernel groups hidden...

▼ 13,3% Memory

100,0% HtoD memcpy

Yani host'tan GPU'ya yapılan kopyalamalar

TensorRT

▼ myelin-exec

Stack Ranges

Start & End

▼ 37,8% Stream 13

Stream 13 sadece Memory işlemleri için kullanılıyor

▶ 100,0% Memory

Compute ve memory paralel işliyor

▼ 0,1% Stream 15

▶ 100,0% Memory

Pipeline hızlanıyor

▶ Threads (20)

NVTX

myelin-exec

▼ [22917] ldconfig.real

0 to 100%

▼ Threads (1)

0 to 100%

[22917] ldconfig.real

